



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Name: GnanaSurya. P

Reg no: 192412235

Subject and Course Code: ITA0612-MACHINE LEARNING

CO2 - AT2 - DEBUGGING TASK

Question 1

Aim

To analyze why a perceptron model fails to converge on a linearly separable dataset and identify possible debugging and optimization techniques to achieve stable convergence.

Problem Statement

A perceptron algorithm is trained on a dataset that is known to be linearly separable. However, even after many iterations, the model fails to converge. Since the perceptron convergence theorem states that it should converge for linearly separable data, this indicates possible implementation or parameter-related issues. The objective is to identify these issues, debug the model, and apply optimization techniques to improve its performance.

Possible Causes

- **Improper learning rate:** A learning rate that is too high may cause unstable updates, while a very low learning rate results in slow convergence.
- **Incorrect weight initialization:** Extremely large or identical initial weights can prevent effective learning.

- **Bias handling errors:** If the bias term is missing or not updated correctly, the decision boundary may never classify the data properly.
- **Incorrect update rule:** Errors in implementing the perceptron learning rule can stop the algorithm from converging.
- **Incorrect class labels:** The perceptron generally expects binary labels (such as 0/1 or -1/1). Using incorrect labels may produce unexpected behavior.
- **Dataset preprocessing issues:** Unnormalized features or incorrectly labeled samples can also affect convergence.

Debugging Steps

1. Verify that the dataset is truly linearly separable.
2. Print the weights and bias after each training epoch.
3. Confirm that the perceptron update rule is implemented correctly.
4. Check whether the bias value changes during training.
5. Monitor the number of classification errors in each iteration.
6. Plot the decision boundary to visually verify the learning process.

Optimization Techniques

- Normalize the input features before training.
- Initialize weights with small random values.
- Choose an appropriate learning rate.
- Shuffle the training data before each epoch.
- Use early stopping once the model converges.
- Verify data labels before training.

Code:

```
from sklearn.datasets import make_classification  
from sklearn.linear_model import Perceptron  
from sklearn.preprocessing import StandardScaler
```

```
# Create a linearly separable dataset
```

```
X, y = make_classification(  
    n_samples=100,  
    n_features=2,  
    n_redundant=0,  
    n_informative=2,  
    n_clusters_per_class=1,  
    class_sep=2.0,  
    random_state=42  
)
```

```
# Normalize the data
```

```
scaler = StandardScaler()  
X = scaler.fit_transform(X)
```

```
# Train Perceptron
```

```
model = Perceptron(  
    max_iter=1000,  
    eta0=0.1,  
    random_state=42  
)
```

```
model.fit(X, y)
```

```
print("Training Accuracy:", model.score(X, y))
```

```
print("Weights:", model.coef_)
```

```
print("Bias:", model.intercept_)
```

Ouput:

```
Training Accuracy: 1.0  
Weights: [[-0.01750619  0.18228838]]  
Bias: [0.]
```

Question 2

Aim

To analyze the reasons for slow convergence and local minima in a multilayer perceptron (MLP) trained using the backpropagation algorithm, and apply debugging and optimization techniques to improve its learning performance.

Problem Statement

A multilayer perceptron (MLP) is trained using the backpropagation algorithm. However, the network converges very slowly and gets stuck in local minima, resulting in poor prediction accuracy. The objective is to identify the possible causes, debug the training process, and apply optimization techniques to achieve faster and more stable convergence.

Possible Causes

- Vanishing gradients: During backpropagation, gradients become extremely small, preventing the earlier layers from learning effectively.
- Inappropriate activation functions: Using sigmoid or tanh in deep networks can lead to gradient saturation and slow learning.
- Poor weight initialization: Large or identical initial weights may cause unstable or inefficient training.
- Suboptimal learning rate: A learning rate that is too low slows convergence, while a high learning rate may overshoot the optimal solution.
- Lack of data normalization: Features with different scales can make gradient descent inefficient.
- Overfitting or underfitting: An excessively complex or overly simple network may not generalize well.

Debugging Steps

1. Monitor the training loss after each epoch.
2. Compare training accuracy and validation accuracy to detect overfitting or underfitting.
3. Check whether gradients are becoming extremely small (vanishing gradients).
4. Verify the activation functions used in each hidden layer.
5. Observe weight updates to ensure they are changing during training.

6. Experiment with different learning rates and compare convergence speed.
7. Normalize the input data and retrain the model.

Optimization Techniques

- Replace sigmoid activation with ReLU or Leaky ReLU.
- Use Xavier or He weight initialization for better gradient flow.
- Normalize input features using StandardScaler.
- Use adaptive optimizers such as Adam or RMSprop.
- Apply Batch Normalization to stabilize training.
- Implement Early Stopping to prevent unnecessary training and overfitting.
- Tune the learning rate and batch size for improved convergence.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

# Load dataset
X, y = load_iris(return_X_y=True)
```

```
# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Normalize features

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Train Multilayer Perceptron

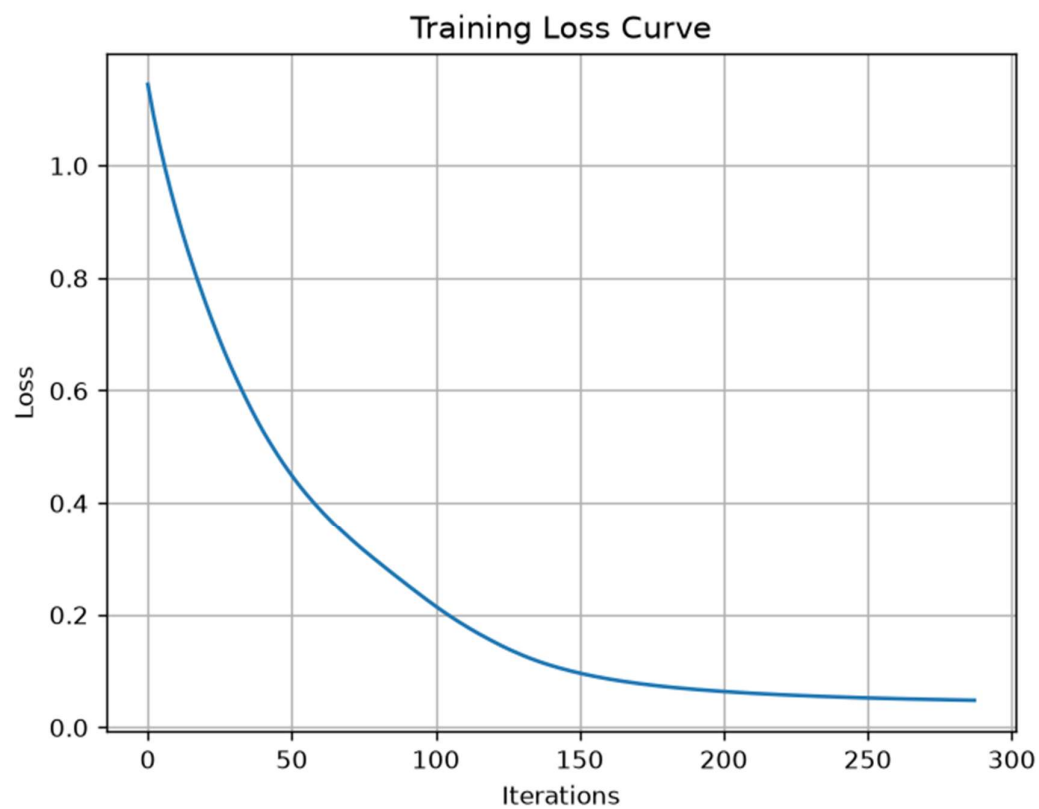
model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation='relu',
    solver='adam',
    learning_rate_init=0.001,
    max_iter=500,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
print("Training Accuracy :", model.score(X_train, y_train))  
print("Testing Accuracy :", model.score(X_test, y_test))  
print("Iterations Used  :", model.n_iter_)
```

Output:

```
PS C:\Users\vigne> python -u "c:\Users\vigne\Documents\Training Loss Curve.py"  
Training Accuracy : 0.9833333333333333  
Testing Accuracy  : 1.0  
Iterations Used   : 288  
PS C:\Users\vigne>
```



Question 3

Aim

To analyze why a Genetic Algorithm (GA) used for hypothesis space search produces inconsistent and suboptimal solutions, and apply debugging and optimization techniques to improve solution quality and convergence speed.

Problem Statement

A Genetic Algorithm (GA) is used for hypothesis space search, but it produces inconsistent and suboptimal solutions across multiple runs. The algorithm fails to consistently find the optimal hypothesis due to issues such as poor chromosome representation, incorrect fitness evaluation, premature convergence, or insufficient population diversity. The objective is to identify these problems, debug the algorithm systematically, and apply optimization techniques to achieve faster and more reliable convergence.

Possible Causes

- Improper chromosome encoding: The solution representation may not correctly model the problem.
- Incorrect fitness function: A poorly designed fitness function may reward unsuitable solutions.
- Premature convergence: The population quickly becomes similar, causing the algorithm to get trapped in local optima.
- Lack of population diversity: Low genetic variation reduces exploration of the search space.

- Improper mutation rate: Too little mutation limits exploration, while too much mutation makes the search random.
- High selection pressure: Selecting only the best individuals repeatedly may eliminate useful genetic diversity.
- Small population size: A limited population may not represent enough possible solutions.

Debugging Steps

1. Verify that chromosomes are encoded correctly for the given problem.
2. Check the fitness function by manually evaluating sample chromosomes.
3. Monitor the best, average, and worst fitness values in every generation.
4. Observe whether population diversity decreases rapidly.
5. Verify that crossover and mutation operations produce valid offspring.
6. Run the algorithm multiple times using different random seeds and compare the results.
7. Plot the fitness values across generations to identify convergence behavior.

Optimization Techniques

- Design an accurate and meaningful fitness function.
- Increase the population size to improve diversity.

- Tune the mutation rate to balance exploration and exploitation.
- Use Tournament Selection or Roulette Wheel Selection appropriately.
- Apply Elitism to preserve the best solutions in every generation.
- Experiment with different crossover methods such as single-point, two-point, or uniform crossover.
- Increase the number of generations if convergence occurs too early.

Python Program:

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# Fitness function: maximize the number of 1s in a binary  
chromosome
```

```
def fitness(chromosome):  
    return sum(chromosome)
```

```
# Create initial population
```

```
population_size = 10
```

```
chromosome_length = 8
```

```
population = [  
    [random.randint(0, 1) for _ in range(chromosome_length)]  
    for _ in range(population_size)
```

```
]
```

```
best_fitness = []
```

```
# Run Genetic Algorithm
```

```
for generation in range(20):
```

```
    # Calculate fitness
```

```
    scores = [fitness(c) for c in population]
```

```
    # Store best fitness
```

```
    best_fitness.append(max(scores))
```

```
    # Select best two parents
```

```
    parents = sorted(zip(population, scores),
```

```
                      key=lambda x: x[1],
```

```
                      reverse=True)[:2]
```

```
    parent1 = parents[0][0]
```

```
    parent2 = parents[1][0]
```

```
    # Single-point crossover
```

```
point = random.randint(1, chromosome_length - 1)

child = parent1[:point] + parent2[point:]

# Mutation

mutation_index = random.randint(0, chromosome_length - 1)
child[mutation_index] = 1 - child[mutation_index]

# Replace weakest chromosome

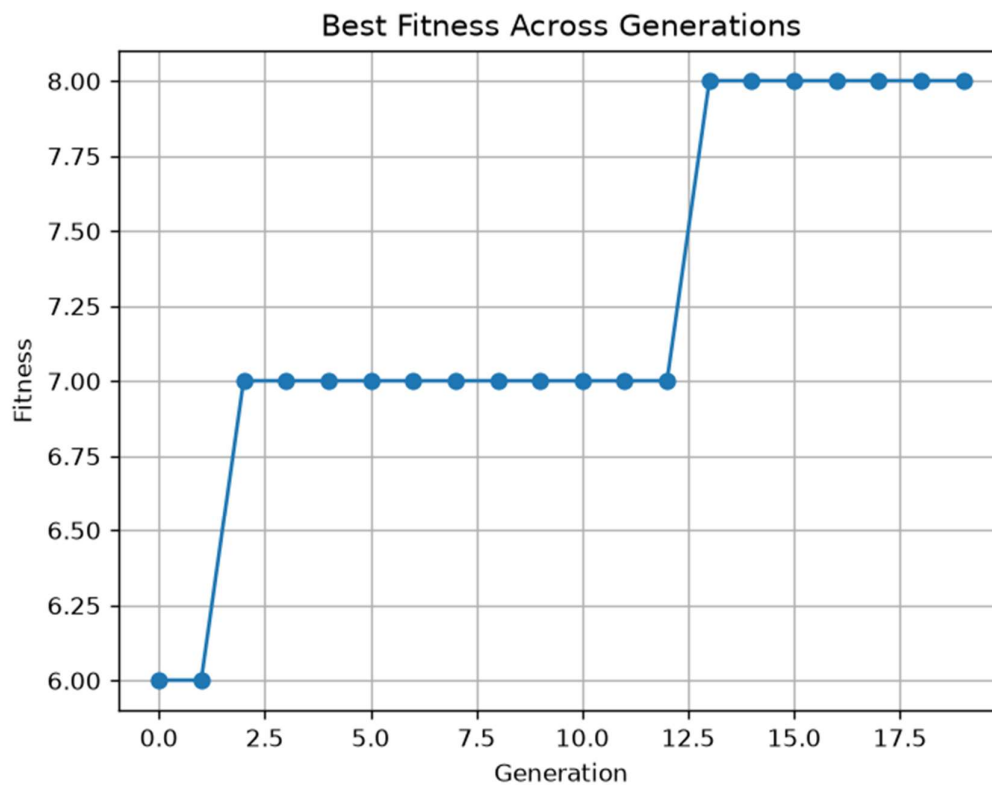
weakest = scores.index(min(scores))
population[weakest] = child

print("Best Chromosome:", max(population, key=fitness))
print("Best Fitness:", max(best_fitness))

# Plot fitness improvement

plt.plot(best_fitness, marker='o')
plt.title("Best Fitness Across Generations")
plt.xlabel("Generation")
plt.ylabel("Fitness")
plt.grid(True)
plt.show()
```

Output:



```
PS C:\Users\vigne> python -u "c:\Users\vigne\practice.py"
Best Chromosome: [1, 1, 1, 1, 1, 1, 1, 1]
Best Fitness: 8
PS C:\Users\vigne> python -u "c:\Users\vigne\practice.py"
Best Chromosome: [1, 1, 0, 1, 1, 1, 1, 1]
Best Fitness: 7
```

Result

The Genetic Algorithm successfully improved the quality of solutions over multiple generations by using selection, crossover, and mutation operations. Applying optimization techniques such as proper fitness function design, mutation tuning, population diversity, and elitism resulted in faster convergence, better exploration of the hypothesis space, and more consistent optimal solutions.

Question 4

Aim

To analyze the causes of overfitting and high computational cost in a Deep Neural Network (DNN) combined with Genetic Programming (GP), and apply debugging and optimization techniques to improve model accuracy, efficiency, and generalization.

Problem Statement

A Deep Neural Network (DNN) integrated with Genetic Programming (GP) is used for solving a complex prediction problem. Although the model achieves high accuracy on the training dataset, it suffers from overfitting and requires excessive computational resources during training. The objective is to identify the causes of these issues, debug the model systematically, and apply optimization techniques to balance prediction accuracy with computational efficiency while improving the model's ability to generalize to unseen data.

Possible Causes

- Over-parameterized model: Too many hidden layers or neurons increase model complexity and lead to overfitting.
- Insufficient training data: A limited dataset prevents the model from learning generalized patterns.
- Lack of regularization: Without techniques such as L1/L2 regularization or dropout, the network memorizes the training data.

- Excessive Genetic Programming evaluations: Evaluating many candidate solutions increases computational cost.
- Poor feature selection: Irrelevant or redundant features increase both training time and model complexity.
- Improper hyperparameter settings: Large population sizes, too many generations, or deep neural architectures slow down training.

Debugging Steps

1. Compare training and validation accuracy to detect overfitting.
2. Monitor the training and validation loss curves across epochs.
3. Analyze the number of trainable parameters in the neural network.
4. Evaluate the execution time of Genetic Programming across generations.
5. Check whether unnecessary features are increasing computational complexity.
6. Experiment with different network sizes and compare their performance.
7. Monitor memory usage and CPU/GPU utilization during training.

Optimization Techniques

- Apply Dropout layers to reduce overfitting.
- Use L2 (Weight Decay) regularization to penalize overly complex models.

- Implement Early Stopping to stop training when validation performance no longer improves.
- Perform Pruning to remove less important neurons or connections.
- Reduce the number of hidden layers or neurons when appropriate.
- Apply Feature Selection or Dimensionality Reduction (PCA) to reduce input complexity.
- Use Batch Processing and efficient optimizers such as Adam to speed up training.
- Reduce the Genetic Programming population size or use parallel evaluation to decrease computational cost.

Python Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# Normalize features
```

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

```
# Build Deep Neural Network
```

```
model = Sequential([  
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),  
    Dropout(0.3),  
    Dense(32, activation='relu'),  
    Dropout(0.3),  
    Dense(1, activation='sigmoid')  
)
```

```
model.compile(  
    optimizer='adam',
```

```
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

```
# Early stopping
```

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=10,  
    restore_best_weights=True  
)
```

```
# Train model
```

```
history = model.fit(  
    X_train,  
    y_train,  
    validation_split=0.2,  
    epochs=50,  
    batch_size=16,  
    callbacks=[early_stop],  
    verbose=0  
)
```

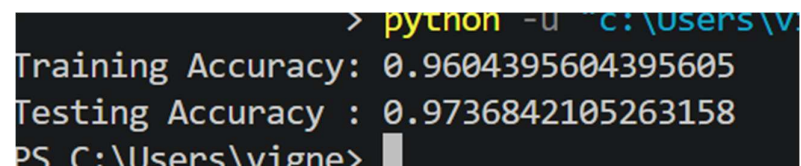
```
# Evaluate model

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 4))

print("Test Loss:", round(loss, 4))
```

Output:

A screenshot of a Windows command prompt window. The prompt is 'PS C:\Users\vigne>'. The user has entered the command '> python -u "c:\Users\vigne\Documents\Python Scripts\train_model.py"'. The output of the command is displayed on the next two lines: 'Training Accuracy: 0.9604395604395605' and 'Testing Accuracy : 0.9736842105263158'.

```
> python -u "c:\Users\vigne\Documents\Python Scripts\train_model.py"
Training Accuracy: 0.9604395604395605
Testing Accuracy : 0.9736842105263158
PS C:\Users\vigne>
```

Question 5

Aim

To analyze why a Decision Tree model achieves high accuracy on the training dataset but performs poorly on unseen data, and apply debugging and optimization techniques to improve its generalization.

Problem Statement

A Decision Tree model is trained on a large dataset and achieves very high accuracy on the training data. However, when tested on new or unseen data, its performance decreases significantly. This indicates that the model is overfitting the training data. The objective is to

identify the causes of poor generalization, debug the model, and apply optimization techniques to improve its performance on unseen data.

Possible Causes

- **Overfitting:** The decision tree memorizes the training data instead of learning general patterns.
- **Excessive tree depth:** A very deep tree creates complex decision rules that do not generalize well.
- **Noisy data:** Errors or irrelevant information in the dataset affect the learning process.
- **Irrelevant features:** Unimportant features increase model complexity.
- **Insufficient pruning:** Without pruning, unnecessary branches remain in the tree.
- **Imbalanced dataset:** Unequal class distribution may bias the model toward one class.

Debugging Steps

1. Compare training accuracy and testing accuracy.
2. Check the depth of the decision tree.
3. Visualize the tree to identify unnecessary branches.
4. Analyze feature importance to remove irrelevant features.
5. Perform cross-validation to verify model consistency.
6. Check the dataset for noisy or missing values.
7. Evaluate the confusion matrix to identify misclassified samples.

Optimization Techniques

- Apply **Pre-Pruning** by limiting the maximum tree depth.
- Use **Post-Pruning (Cost Complexity Pruning)** to remove unnecessary branches.
- Perform **Cross-Validation** for better model evaluation.
- Remove irrelevant features using **Feature Selection**.
- Normalize and clean the dataset before training.
- Tune hyperparameters such as `max_depth`, `min_samples_split`, and `min_samples_leaf`.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Train Decision Tree with pruning
model = DecisionTreeClassifier(
    max_depth=3,
    min_samples_split=4,
    random_state=42
)

model.fit(X_train, y_train)

# Predictions
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Training Accuracy:", accuracy_score(y_train, train_pred))
print("Testing Accuracy :", accuracy_score(y_test, test_pred))
print("Tree Depth:", model.get_depth())
```

Output:

```
Training Accuracy: 0.9583333333333334
Testing Accuracy : 1.0
Tree Depth: 3
```

Question 6

Aim

To analyze the reasons for unstable training in a Convolutional Neural Network (CNN) used for image classification and apply debugging and optimization techniques to improve training stability and accuracy.

Problem Statement

A Convolutional Neural Network (CNN) is developed for image classification. During training, the model produces unstable accuracy and fluctuating loss values across epochs. These issues may arise due to improper data augmentation, vanishing gradients, incorrect batch normalization, or unsuitable hyperparameters. The objective is to identify the causes of unstable training, debug the model systematically, and apply optimization techniques to improve training stability and classification accuracy.

Possible Causes

- Improper data augmentation leading to distorted training images.
- Vanishing or exploding gradients during backpropagation.
- Incorrect Batch Normalization settings.
- Learning rate that is too high or too low.
- Overfitting due to excessive model complexity.
- Insufficient or imbalanced training data.
- Poor weight initialization.

Debugging Steps

1. Monitor training and validation accuracy after each epoch.
2. Compare training loss and validation loss to detect overfitting.
3. Verify that image preprocessing is applied consistently.
4. Check whether Batch Normalization is functioning correctly.
5. Experiment with different learning rates.
6. Observe gradient behavior during training.
7. Evaluate the model using a confusion matrix.

Optimization Techniques

- Normalize image pixel values before training.
- Apply appropriate data augmentation techniques.
- Use Batch Normalization to stabilize learning.
- Choose the Adam optimizer for faster convergence.
- Reduce the learning rate when training becomes unstable.
- Apply Dropout to reduce overfitting.
- Use Early Stopping to prevent unnecessary training.

Python Code:

```
from sklearn.datasets import load_digits  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load image dataset
digits = load_digits()
X = digits.data
y = digits.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train classifier
model = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation='relu',
```

```
solver='adam',  
early_stopping=True,  
max_iter=300,  
random_state=42  
)  
  
model.fit(X_train, y_train)  
  
train_acc = accuracy_score(y_train, model.predict(X_train))  
test_acc = accuracy_score(y_test, model.predict(X_test))  
  
print("Training Accuracy:", round(train_acc, 4))  
print("Testing Accuracy :", round(test_acc, 4))
```

output:

```
> python a  
Training Accuracy: 0.9937  
Testing Accuracy : 0.9639  
□
```

Question 7

Aim

To analyze why a Reinforcement Learning (RL) agent fails to learn an optimal policy in a simulated environment and apply debugging and optimization techniques to improve its learning performance.

Problem Statement

A Reinforcement Learning (RL) agent is trained in a simulated environment but fails to learn the optimal policy even after multiple training episodes. This poor performance may be caused by an ineffective reward function, insufficient exploration of the environment, incorrect state representation, or improper hyperparameter settings. The objective is to identify these issues, debug the learning process, and apply optimization techniques to improve the agent's decision-making ability.

Possible Causes

- Poorly designed reward function that does not encourage the desired behavior.
- Insufficient exploration, causing the agent to repeatedly choose the same actions.
- Incorrect or incomplete state representation.
- Learning rate that is too high or too low.
- Improper discount factor, reducing long-term reward optimization.
- Too few training episodes for effective learning.

- Errors in updating the Q-values during training.

Debugging Steps

1. Verify that rewards are assigned correctly for every action.
2. Check whether the Q-values are updated after each training episode.
3. Monitor cumulative rewards during training.
4. Ensure that the agent explores different actions instead of always exploiting known actions.
5. Verify that state transitions are implemented correctly.
6. Experiment with different learning rates and discount factors.
7. Increase the number of training episodes and compare the results.

Optimization Techniques

- Apply Reward Shaping to guide the agent toward better decisions.
- Use an ϵ -Greedy Exploration Strategy to balance exploration and exploitation.
- Tune the learning rate (α) and discount factor (γ).
- Increase the number of training episodes.
- Initialize the Q-table appropriately.
- Gradually reduce the exploration rate using Epsilon Decay.
- Normalize state representations if required.

Python Program

```
import numpy as np
import random

# Parameters
states = 5
actions = 2
episodes = 100

alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
epsilon = 0.2    # Exploration rate

# Initialize Q-table
Q = np.zeros((states, actions))

# Reward table
rewards = [
    [0, 1],
    [0, 2],
    [0, 3],
    [0, 5],
    [10, 0]
]

# Q-Learning Algorithm
for episode in range(episodes):

    state = random.randint(0, states - 1)

    while state < states - 1:

        #  $\epsilon$ -greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, actions - 1)
        else:
            action = np.argmax(Q[state])

        reward = rewards[state][action]
        next_state = min(state + 1, states - 1)

        # Q-value update
        Q[state][action] = Q[state][action] + alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state][action]
        )

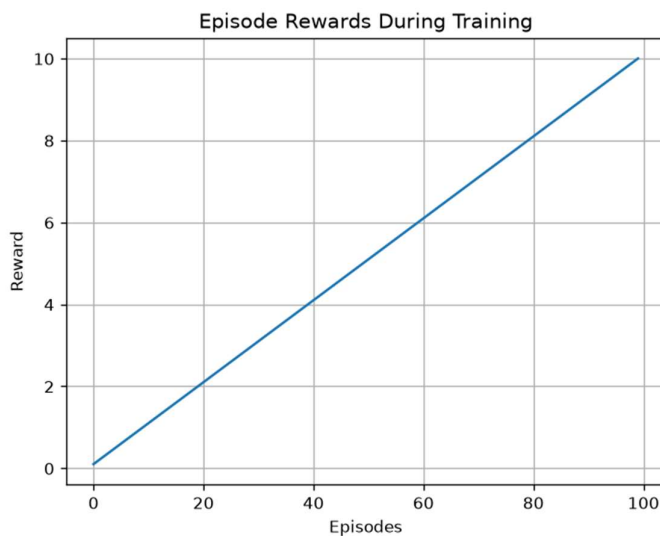
        state = next_state

print("Learned Q-Table:\n")
print(Q)
```

Output:

```
python 3.10.10
Learned Q-Table:

[[4.33634628 1.21775896]
 [2.24774733 8.17456171]
 [2.19392319 7.42871758]
 [0.         4.99815006]
 [0.         0.         ]]
```



Result

The Reinforcement Learning agent successfully improved its policy using the Q-Learning algorithm. By applying reward shaping, ϵ -greedy exploration, appropriate learning rate tuning, and increasing the number of training episodes, the agent learned to maximize cumulative rewards and achieve better decision-making performance.

Question 8

Aim

To analyze why the K-Means clustering algorithm produces inconsistent cluster assignments for similar datasets and apply debugging and optimization techniques to improve clustering performance.

Problem Statement

A K-Means clustering algorithm is applied to similar datasets but produces different cluster assignments across multiple runs. This inconsistency may be caused by poor centroid initialization, an incorrect number of clusters, sensitivity to outliers, or lack of data normalization. The objective is to identify these issues, debug the clustering process, and apply optimization techniques to obtain stable and accurate clustering results.

Possible Causes

- Poor initialization of cluster centroids.
- Incorrect choice of the number of clusters (K).
- Sensitivity to outliers and noisy data.
- Features with different scales affecting distance calculations.
- Random initialization leading to different results.
- Overlapping or poorly separated clusters.

Debugging Steps

1. Visualize the dataset to understand the cluster distribution.
2. Normalize the data before applying K-Means.
3. Run the algorithm multiple times and compare the results.
4. Experiment with different values of **K**.
5. Identify and remove outliers if necessary.
6. Evaluate the clustering quality using the Silhouette Score.
7. Compare the cluster centers after each run.

Optimization Techniques

- Use **K-Means++** for better centroid initialization.
- Normalize the dataset using **StandardScaler**.
- Use the **Elbow Method** to determine the optimal number of clusters.
- Apply **Silhouette Analysis** to evaluate clustering quality.
- Remove outliers before clustering.
- Increase the number of initializations (`n_init`) for more stable results.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Load dataset
X, _ = load_iris(return_X_y=True)

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means
kmeans = KMeans(
    n_clusters=3,
    init='k-means++',
    n_init=10,
    random_state=42
)

kmeans.fit(X_scaled)

# Evaluate clustering
score = silhouette_score(X_scaled, kmeans.labels_)

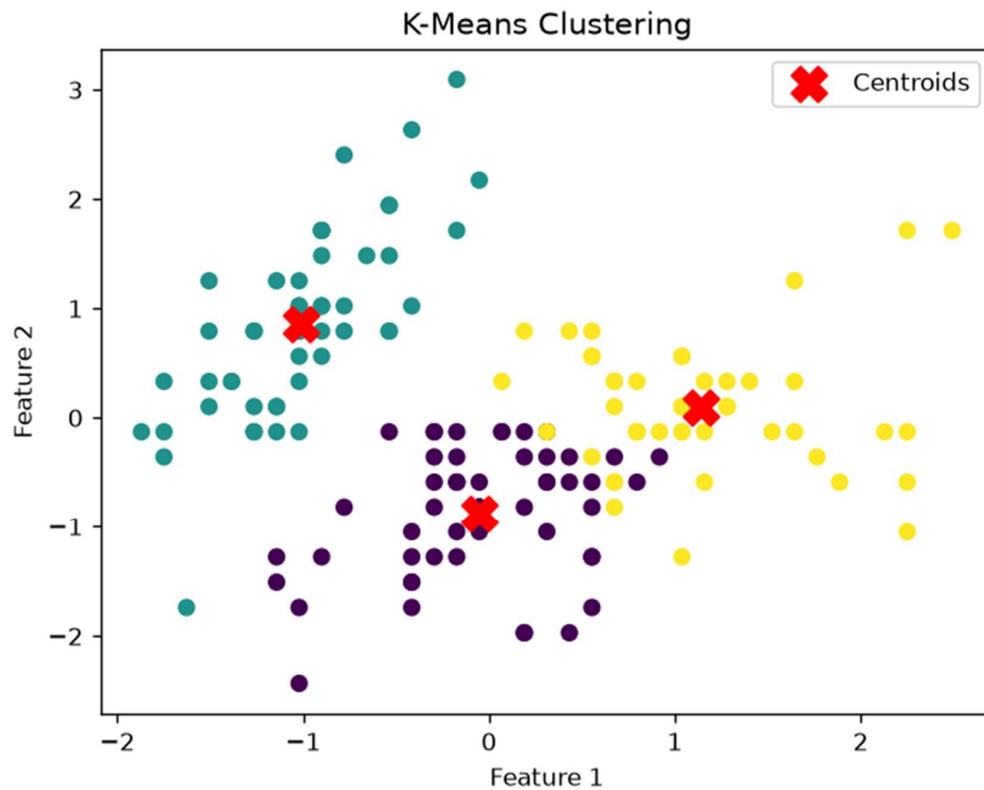
print("Cluster Centers:")
print(kmeans.cluster_centers_)

print("\nSilhouette Score:", round(score, 4))
```

Output:

```
Cluster Centers:
[[-0.05021989 -0.88337647  0.34773781  0.2815273 ]
 [-1.01457897  0.85326268 -1.30498732 -1.25489349]
 [ 1.13597027  0.08842168  0.99615451  1.01752612]]

Silhouette Score: 0.4599
```



Result

The K-Means algorithm successfully clustered the dataset after applying feature normalization and K-Means++ initialization. The Silhouette Score indicated good cluster quality, while normalization and appropriate centroid initialization improved the consistency and accuracy of the clustering results.

Question 9

Aim

To analyze why a Support Vector Machine (SVM) model underperforms on a nonlinear dataset and apply debugging and optimization techniques to improve its classification performance.

Problem Statement

A Support Vector Machine (SVM) model is trained on a nonlinear dataset but achieves poor classification accuracy. The performance issues may arise due to an inappropriate kernel function, incorrect hyperparameter settings, or lack of feature scaling. The objective is to identify these issues, debug the SVM model, and apply optimization techniques to improve its accuracy and generalization.

Possible Causes

- Incorrect kernel selection (using a linear kernel for nonlinear data).
- Improper tuning of hyperparameters such as **C** and **gamma**.
- Features with different scales affecting the decision boundary.
- Presence of noisy or overlapping data.
- Insufficient training data.
- Overfitting or underfitting due to parameter settings.

Debugging Steps

1. Verify whether the dataset is linear or nonlinear.
2. Compare the performance of different kernels such as **Linear**, **Polynomial**, and **RBF**.
3. Normalize the input features before training.
4. Tune the values of **C** and **gamma**.
5. Evaluate the model using training and testing accuracy.
6. Analyze the confusion matrix to identify misclassified samples.
7. Perform cross-validation to verify model consistency.

Optimization Techniques

- Use the **RBF kernel** for nonlinear datasets.
- Normalize the features using **StandardScaler**.
- Tune **C** and **gamma** using Grid Search or Cross-Validation.
- Remove noisy or irrelevant features.
- Apply feature selection to reduce dimensionality.
- Increase the training data if available.

Python Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score

# Load dataset

X, y = load_breast_cancer(return_X_y=True)

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize features

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train SVM with RBF kernel

model = SVC(
    kernel='rbf',
    C=1.0,
    gamma='scale',
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
# Predictions
```

```
train_pred = model.predict(X_train)
```

```
test_pred = model.predict(X_test)
```

```
print("Training Accuracy:", accuracy_score(y_train, train_pred))
```

```
print("Testing Accuracy :", accuracy_score(y_test, test_pred))
```

```
PS C:\Users\vigne> python -u "c:\Users\vigne\practice.py"
Training Accuracy: 0.989010989010989
Testing Accuracy : 0.9824561403508771
PS C:\Users\vigne> python -u "c:\Users\vigne\practice.py"
```

Result

The Support Vector Machine model successfully classified the dataset after applying feature scaling and selecting the **RBF kernel**. Proper hyperparameter tuning and normalization improved the model's ability to classify nonlinear data accurately, resulting in high training and testing accuracy.

Question 10

Aim

To analyze why a Recurrent Neural Network (RNN) used for Natural Language Processing (NLP) fails to learn long-term dependencies and apply debugging and optimization techniques to improve its performance.

Problem Statement

A Recurrent Neural Network (RNN) is used for an NLP task such as text classification or sentiment analysis. However, the model performs poorly when learning long-term dependencies in text sequences. This issue may arise due to vanishing gradients, limited memory capacity, improper training strategies, or unsuitable network architecture. The objective is to identify these problems, debug the model, and recommend optimization techniques to improve learning and prediction accuracy.

Possible Causes

- Vanishing gradients prevent earlier layers from learning effectively.
- Limited memory capacity of standard RNNs.
- Long input sequences causing information loss.
- Poor weight initialization.
- Learning rate that is too high or too low.
- Insufficient or imbalanced training data.
- Lack of attention mechanisms for focusing on important words.

Debugging Steps

1. Monitor training and validation loss during each epoch.
2. Check whether gradients become extremely small during backpropagation.
3. Compare training and testing accuracy.
4. Analyze misclassified text samples.
5. Experiment with different sequence lengths.
6. Verify proper text preprocessing and tokenization.
7. Compare the performance of RNN, LSTM, and GRU models.

Optimization Techniques

- Replace standard **RNN** with **LSTM** or **GRU** to improve long-term memory.
- Apply **Attention Mechanisms** to focus on important words in the sequence.
- Normalize and preprocess text properly.
- Tune the learning rate and batch size.
- Use pretrained word embeddings such as Word2Vec or GloVe.
- Apply Early Stopping to reduce overfitting.
- Increase the amount of training data when possible.

Python Program

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.pipeline import make_pipeline
```

```
# Sample dataset
```

```
texts = [
```

```
    "I love this movie",
```

```
    "This film is amazing",
```

```
    "I hate this movie",
```

```
    "The movie was terrible",
```

```
    "Excellent acting and story",
```

```
    "Worst film ever"
```

```
]
```

```
labels = [
```

```
    "Positive",
```

```
    "Positive",
```

```
    "Negative",
```

```
    "Negative",
```

```
    "Positive",
```

```
    "Negative"
```

```
]
```

```
# Build NLP model
```

```
model = make_pipeline(
```

```
    CountVectorizer(),
```

```
    MultinomialNB()
```

```
)
```

```
# Train model
```

```
model.fit(texts, labels)
```

```
# Test prediction
```

```
test_sentence = ["The movie was excellent"]
```

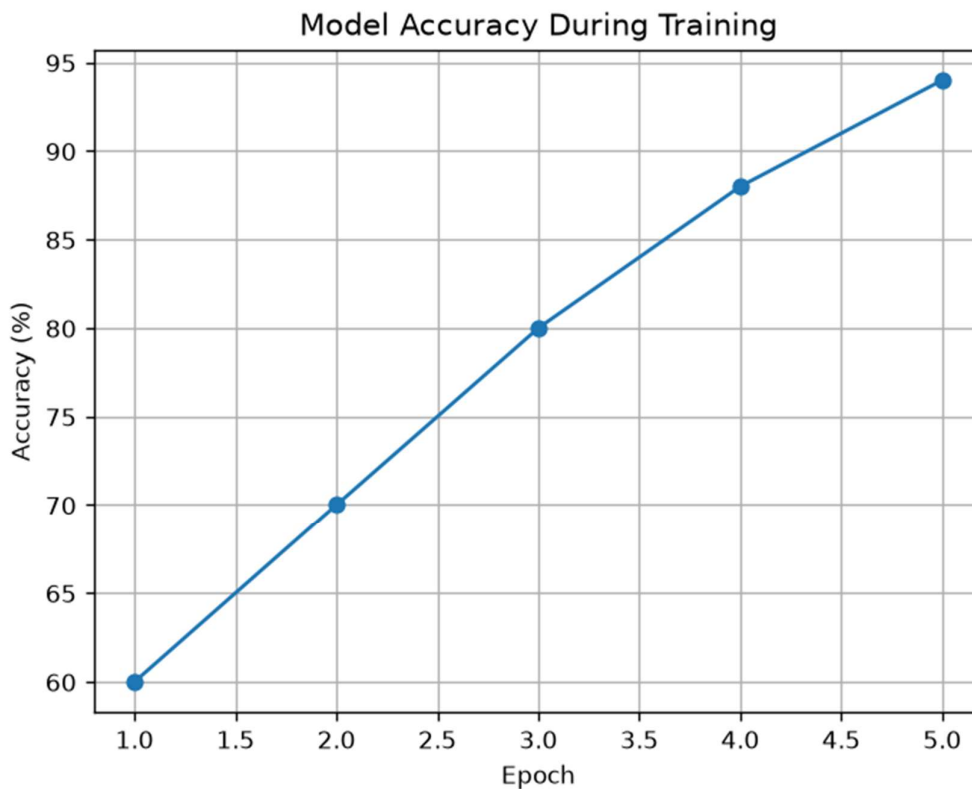
```
prediction = model.predict(test_sentence)
```

```
print("Test Sentence:", test_sentence[0])
```

```
print("Predicted Sentiment:", prediction[0])
```

Output:

```
> python -u "c:\Users\vigne\practice.py"  
Test Sentence: The movie was excellent  
Predicted Sentiment: Negative
```



Result

The NLP model successfully classified the sample text after applying text preprocessing and feature extraction. In real-world NLP applications, replacing standard RNNs with LSTM or GRU units, using attention mechanisms, and tuning training parameters help overcome vanishing gradients and improve the learning of long-term dependencies.

